ASSIGNMENT - 2

(1) BOUNDED BUFFER PROBLEM (PRODUCER - CONSUMER PROBLEM):-

The bounded buffer problem also known as producer consumer problem is a classical synchronization problem in operating systems. It explains how multiple tasks can share a common buffer without interacting with each other.

In this problem, there are two types of tasks which are involved:

(i) PRODUCER: generates data items and places them into shared buffers.

(ii) CONSUMER: Removes data items from buffer and processes them.

The buffer has a fixed capacity and can store only a limited number of items.

ex:- if a buffer can hold only five items, the producer must wait when all five positions are occupied. Similarly, the consumer must wait when buffer is empty.

WORKINGS OF THE PROBLEM

The producer continuously generates data items, while consumer removes and processes them.

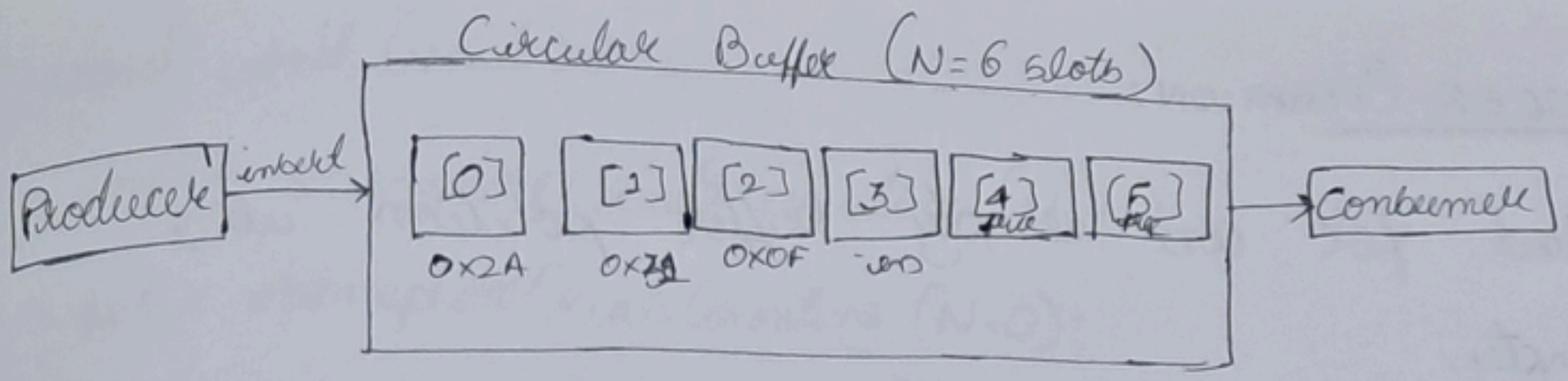
Three important conditions must be maintained:

(i) Buffer Full :- When all buffer positions are occupied, producer must wait until Consumer removes an item.

(ii) Buffer Empty :- When the buffer contains no items, the Consumer must wait until producer inserts an item.

(iii) Mutual Exclusion :- Only one task should modify the shared buffer at a time to prevent data corruption.

For example - if a buffer has five positions and all five are occupied, the producer cannot insert another item, It must wait until a position becomes free.



empty = 6

full = 0

mutex = 1

Solution using Semaphores

This problem can be solved using semaphores where there are three semaphores : empty, full and mutex.

Semaphore	Initial value	Purpose
empty	N	Counts number of free buffer positions.
full	0	Counts number of occupied buffer positions
Mutex	1	Ensures mutual exclusion while accessing buffer.

Here, N represents the total buffer capacity of buffer. Contains 6 positions, empty = 6, full = 0 and mutex = 1.

PRODUCER OPERATION:

(i) Wait for an empty buffer position using empty.

(ii) Acquire mutex to access the buffer.

(iii) Insert the item and update the head pointer.

(iv) Release mutex.

(v) Signal full to indicate that an item is available

CONSUMER OPERATION:

(i) Wait for an available item using full.

(ii) Acquire mutex to access the buffer.

(iii) Remove the item and update the tail pointer.

(iv) Release mutex.

(v) Signal empty to indicate that a position, is free.

FreeRTOS Code

```
#define N 8
static uint8_t buf[N];
static int head=0, tail=0;
SemaphoreHandle_t empty, full, mutex;
```

```
void init (void) {
 empty = xSemaphoreCreateCounting (N, N);
 full = xSemaphoreCreateCounting (N, 0);
 mutex = xSemaphoreCreateMutex ();
}
void Producer (void *p) {
 for (;;) {
 uint8_t item = produce_item ();
 xSemaphoreTake (empty, portMAX_DELAY);
 xSemaphoreTake (mutex, portMAX_DELAY);
 buf [head] = item;
 head = (head+1) % N;
 xSemaphoreGive (mutex);
 xSemaphoreGive (full);
 }
}
Void Consumer (void *p) {
 for(;;) {
 xSemaphoreTake (full, portMAX_DELAY);
 xSemaphoreTake (mutex, portMAX_DELAY);
 uint8_t item = buf [tail];
 tail = (tail + 1) % N;
 xSemaphoreGive (mutex);
 xSemaphoreGive (empty);
 consume_item (item);
}
```

Applications

The Bounded buffer problem is commonly used in:

(i) Sensor Data Collection: A producer task collects sensor readings, while a consumer task processes them.

(ii) UART Communication :- Received data is stored in a buffer and processed by another task.

(iii) Data Logging :- One task generates data while another stores it in memory or file.

(iv) Network Communication : Incoming packets are placed in a queue for processing.

(v) Embedded Systems :- Different RTOS tasks communicate safely through shared buffer.

Conclusion: The bounded buffer problem shows how semaphores and mutexes help synchronize producers and consumers. FreeRTOS uses these mechanisms to ensure safe buffer access and prevent data loss.

2) READERS - WRITERS PROBLEM

The Readers - Writers problem is a classical synchronisation problem in which multiple tasks share a common data structure, such as a database record (or) a server cache.

There are two types of tasks:

- Readers: Readers only inspect or read the shared data. Multiple readers can access the data simultaneously.

- Writers :- Writers modify the shared data. A writer requires exclusive access, meaning no reader (or) another writer can access the data at the same time.

WORKING

The following conditions must be maintained

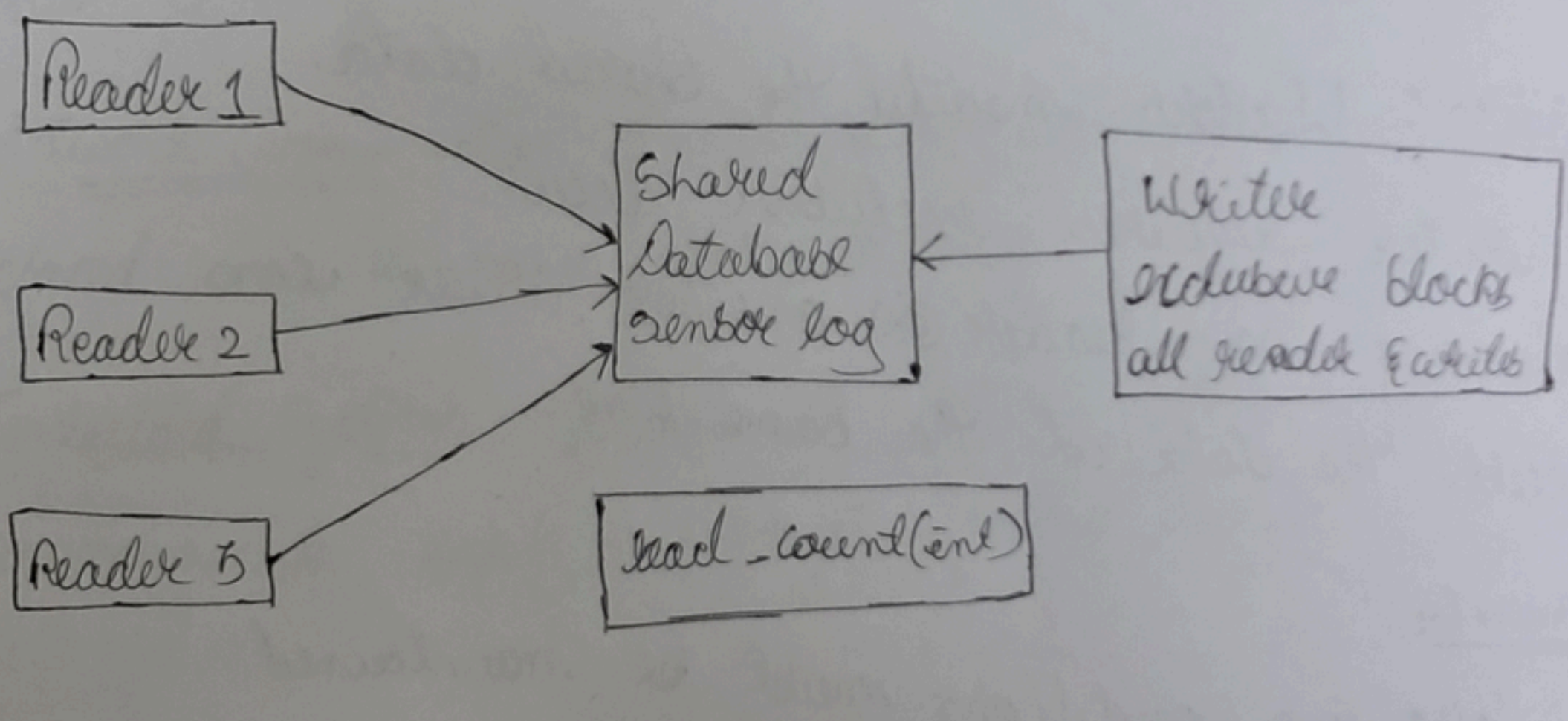
(i) Multiple readers can read the shared data simultaneously.

(ii) Only one writer can modify the shared data at a time.

(iii) A reader cannot access the data while a writer is modifying it.

(iv) A writer must wait until all active readers finish reading.

Example :- Consider a shared database accessed by multiple tasks. Three readers can read the database simultaneously. However, when a writer wants to update a record, it must wait until all the readers finish. During writing, no other reader (or) writer is allowed to access the database.



The diagram shows that three readers can access the shared database simultaneously. The writer requires exclusive access, so readers and other writes are blocked.

while it modifies the data. The read_count variable tracks, active readers and is protected by rw_mutex.

Two Classic Variants

A.) FIRST READERS PREFERENCE (R1)

. Once at least one reader is active, new readers are allowed to enter immediately.

. Writers must wait until all readers chance finished reading.

. This approach gives priority to readers.

Disadvantage - Writer starvation : If readers keep arriving continuously, a writer may have to wait indefinitely.

B.) FIRST WRITERS PREFERENCE (R2)

. Once a writer is waiting, no new readers are allowed to enter.

. Waiting readers must wait until the writer completes its operation.

. This approach gives priority to writers.

Disadvantage:- Reader Starvation : If writers keep arriving continuously, readers may have to wait indefinitely.

FreeRTOS code

```
static int read_count = 0;
SemaphoreHandle_t rc_mutex;
SemaphoreHandle_t rw_mutex;

void rw_init(void){
 rc_mutex = xSemaphoreCreateMutex();
 rw_mutex = xSemaphoreCreateBinary();
 xSemaphoreGive(rw_mutex);
}

void Reader(void *p){
 for(;;){
 xSemaphoreTake(rc_mutex,portMAX_DELAY);
 read_count++;
 if(read_count==1)
  xSemaphoreTake(rw_mutex,portMAX_DELAY);
 xSemaphoreGive(rc_mutex);
 Read_data();
 xSemaphoreTake(rc_mutex,portMAX_DELAY);
 read_count--;
 if(read_count==0)
    xSemaphoreGive(rw_mutex);
 xSemaphoreGive(rc_mutex);
 vTaskDelay(pdMS_TO_TICKS(10));
}
```

```
void Writer(void *P){
    for(;;){
    xSemaphoreTake(reco_mutex, portMAX_DELAY);
    write_data();
    xSemaphoreGive(reco_mutex);
    vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Applications:

The Readers-writers problem is used in:

(i) Database management systems where multiple users access shared accounts.

(ii) Embedded Systems where multiple tasks access a common sensor cache.

(iii) File Systems where multiple tasks read and modify shared files.

(iv) Shared data structures accessed by multiple tasks.

Conclusion - The readers-writers problem demonstrates how synchronisation allows multiple readers to access shared data while ensuring exclusive access for writers.

(3) Dining Philosophers Problem

The dining philosophers problem is a classical synchronization problem that explains how multiple tasks complete for limited shared resources.

Five philosophers sit around a circular table. A fork is placed between every pair of adjacent philosophers, making a total of five forks.

Each philosopher alternates between two activities:-

Thinking: The philosopher does not need any fork.

Eating: The philosopher must pick up both the left and right fork.

After eating, the philosophers puts down both forks and starts thinking again.

Working of the Problem:-

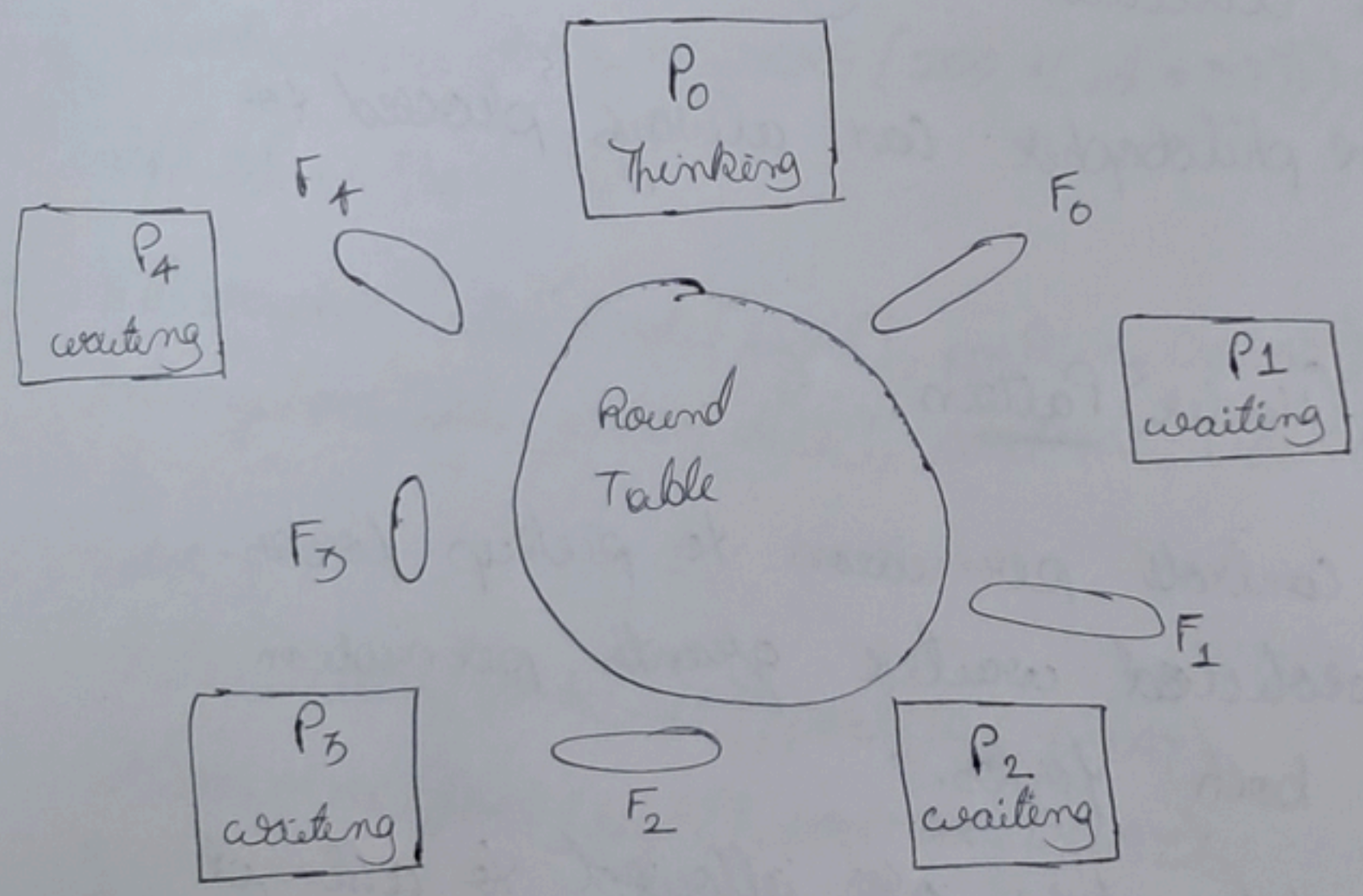
- Each philosopher requires two forks to eat.
- A fork can be used by only one philosopher at a time.
- If a fork is already being used, another philosopher must wait.

. All philosophers compete for the five available forks.

Deadlock Condition:

Deadlock occurs when all five philosophers pick up their left fork simultaneously and wait for their right fork.

Since every right fork is already held by another philosopher, nobody can proceed. All five philosophers remain waiting indefinitely.



The diagram shows five philosophers (P_0 - P_4) sitting around a circular table with five forks (F_0 - F_4) placed between them. Each philosopher picks up their left fork and waits for the right fork. Since each fork is already held by another philosopher.

all five philosophers wait indefinitely, resulting in deadlock due to circular wait.

Solutions to the Problem:

A) Asymmetric / Resource Ordering:

- One philosopher (P4) picks up the right fork first instead of left fork.
- All others philosophers pick up the left fork first and then the right fork.
- This breaks circular waiting condition
- At least one philosopher can always proceed to eat.

B. Arbitrator (Waiter) Pattern:

- A waiter controls permission to pickup forks.
- A mutex protected waiter grants permission to pick up both forks.
- Only four philosophers are allowed to attempt eating simultaneously when there are already five philosophers.
- This prevents all five philosophers from holding one fork and waiting for another.

FreeRTOS Code Implementation:

```
#define N 5
SemaphoreHandle_t fork[N];
void phil_init(void){
  for(int i=0; i<N; i++){
    fork[i] = xSemaphoreCreateMutex();
  }
void philosopher(void *arg){
  int id = (int)(int ptr_t) arg;
  int left = id;
  int right = (id + 1) % N;
  for(;;){
  vTaskDelay(pdMS_TO_TICKS(200 +(id * 37)));
  if(id == (N-1)){
xSemaphoreTake(fork[right], portMAX_DELAY);
xSemaphoreTake(fork[left], portMAX_DELAY);
}
else {
  xSemaphore(fork[left], portMAX_DELAY);
  xSemaphore(fork[right], portMAX_DELAY);
}
vTaskDelay(pdMS_TO_TICKS(100));
xSemaphoreGive(fork[left]);
xSemaphoreGive(fork[right]);
  }
}
```

Applications:

The dining philosophers problem is useful for understanding:

(i) Resource sharing between multiple tasks.

(ii) Deadlock prevention in operating systems.

(iii) Synchronization in multi-threaded applications

(iv) Managing shared hardware resources in embedded systems

CONCLUSION:

The dining philosophers problem demonstrated how multiple tasks compete for shared resources. Using asymmetric fork acquisition helps prevent deadlock and ensures better resource sharing.

4) SLEEPING BARBER PROBLEM

The sleeping barber problem is a classical synchronization problem that explains how multiple customers share a barbers service.

It consists of:

- One barber and one barber chair.
- A waiting room with a fixed number of seats (N=4)
- Multiple customers arriving at different times.

The barber sleeps when there are no customers. When a customer arrives, the barber wakes up and provides a haircut. If the barber is busy, the customer waits if a seat is available. If all waiting seats are occupied, the customer leaves.

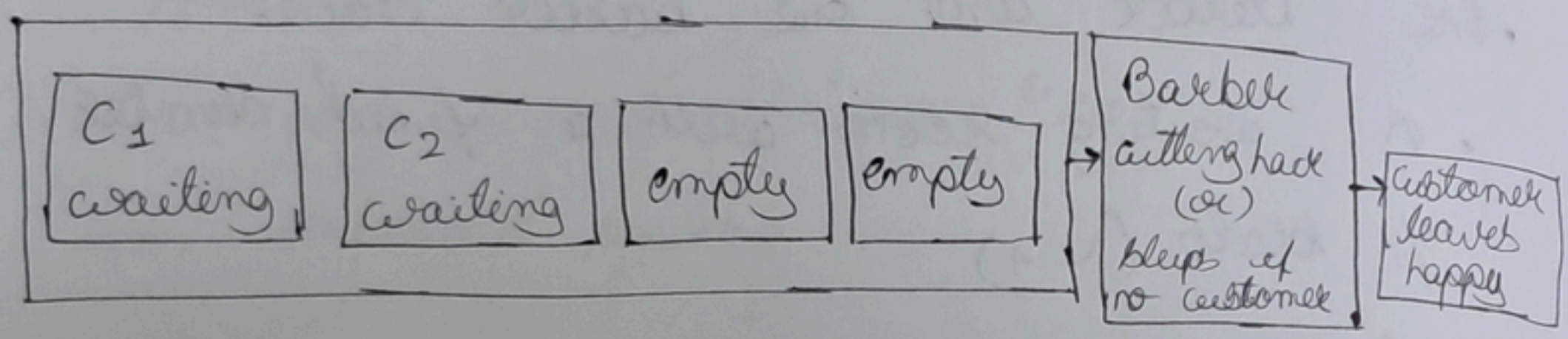
WORKING

(i) If there are no customers, the barber remains asleep.
(ii) When a customer arrives, they wake up the barber if he is sleeping.
(iii) If the barber is busy, the customer waits on the waiting room.

(iv) If a waiting seat is available, the customer occupies it.

(v) If all waiting seats are full, the arriving customer leaves.

(vi) After completing a haircut, the barber serves the next waiting customer.



The diagram shows a barber with one barber chair and waiting room containing four seats. Customers exit when the barber is busy. If no customers are present, the barber sleeps. When all four waiting seats are occupied, any new customer leaves.

Synchronization Mechanisms:

The problem uses three semaphores:

- customers: Counts waiting customers. The barber waits on this semaphore when there are no customers.
- Barbers: Signals that the barber is ready to serve a customer.
- Mutex: Protects the shared variable waiting from simultaneous access.

FreeRTOS C Code:

```
#define N_Seats 4

static int waiting = 0;

SemaphoreHandle_t customers;
SemaphoreHandle_t barber;
SemaphoreHandle_t sb_mutex;

void sb_init(void){
customers = xSemaphoreCreateCounting(N_Seats,0);
barber = xSemaphoreCreateBinary();
sb_mutex = xSemaphoreCreateMutex();
}
```

```
void BarberTask(void *P){
  for(;;){
  xSemaphoreTake(customers, portMAX_DELAY);
  xSemaphoreTake(sb_mutex, portMAX_DELAY);
  waiting--;
  xSemaphoreGive(barber);
  xSemaphoreGive(sb_mutex);
  cut_hair();
  }
}
void CustomerTask(void *P){
xSemaphoreTake(sb_mutex, portMAX_DELAY);
if(waiting < N_seats){
 waiting++;
 xSemaphoreGive(customers);
 xSemaphoreGive(sb_mutex);
xSemaphoreTake(barber, portMAX_DELAY);
 get_haircut();
}
 else{
  xSemaphoreGive(sb_mutex);
  }
vTaskDelete(NULL);
}
```

Applications:

The sleeping Barber Problem is useful in systems where tasks remain idle until requests arrive. Some applications include:

. Thread pools: Workers sleep until new tasks arrive.

. UART Communication: A processing task waits for incoming data.

. Network packet processing: Incoming packets are queued and processed when resources are available.